

Functions (part 2)

CMSC 104 Section 2
September 28 2026

Administrative Notes

Classwork 4 will be released on Wednesday; you will have one week to do it

Project 2 will be released next Monday, October 5 - you will have two weeks to do it.

- Please try to stay ahead of the work - it's much easier that way

Remember that there is no class two weeks from today - October 12

What was the point of last Wednesday's lecture?

Why do we define functions?

- They make it easier to “write a little, test a little”
- They make it more efficient to write code when you have to do the same task multiple times
- They make it easier to reuse code you've already written - or somebody else has already written.

How does Python execute a program?

Python looks for the “if `__name__` == “`__main__`”:

 statement, and starts executing from the first statement after that

- Functions are only executed if and when they are called. Otherwise they're ignored

What happens if there is no “if `__name__` == “`__main__`” statement in the program?

- If no functions are defined, Python just assumes that the entire program is the main program and starts executing from the first line of code it encounters.
- If functions are defined, nothing happens

Wait, you can write a program with only functions?

Yep. See the example.

That's a complete

Python program. When

You run it nothing happens.

```
def calc_factorial(num):  
    product = 1  
    for i in range(1,num+1):  
        product *= i  
    return product  
  
def print_message():  
    print("We're proving this will work")
```

Why would you do such a thing?

Because you want to import the functions into other programs you will write later.

In other words, you want to reuse your code.

We'll look at some examples of that.

A key point:

- If you import a file you wrote, you can reuse any function from that file
- You CANNOT reuse code that's in the main program!!!
-

Now the stuff we didn't finish last week

The return statement

return is the statement used to pass values from the function back to the main program (or back to the calling function)

The syntax is

return variable_name # you can optionally put the variable name in ()

For this class, a function will return one value.

Using returned values

If you want to use the value that's returned, you have to do so in the calling program/function

If the function contains:

```
def factorial(num):  
    Product = 1  
    For i in range(num):  
        Product *= i  
    return product
```

The calling program should use the value:

```
fact = factorial(5)
```

Or

```
sum = factorial(7) + factorial(2)
```

or...

Example - calculate the “fourth root” of a positive number

```
# the “fourth root” of a number is that number raised to the ( $\frac{1}{4}$ ) power
```

```
#
```

```
def fourth_root(num):
```

```
    ans = num**(1/4)
```

```
    return (ans)
```

```
#set the variable's value, then call the function
```

```
original_number = 81
```

```
final_number = fourth_root(original_number)
```

```
print(final_number)
```

Program control in a function

A function stops executing once it executes a `return ()` statement. Let's look at `fourth_root()` again. This time we'll add some code after the “`return()`” statement

the “fourth root” is the number raised to the $\frac{1}{4}$ power

```
def fourth_root(num):
```

```
    ans = num**(1/4)
```

```
    return (ans)
```

The following statement is not going to execute, because the function has already ended

due to the “return” statement

```
    print(“the code has successfully executed”)
```

The value “None”

Python defines a special value “None” which is of type “Nonetype”

If a function does not otherwise return a value, it returns “None”

- Functions do not have to contain a return statement. Any function without a return statement returns “None.”
- If a function’s return statement is not executed, the function returns “None.”
- You can explicitly tell the function to return “None.”

A common error is to have a function return None when you expected it to return something else.

- You’ll see an error message like:
- `Error; type ‘Nonetype’ is not iterable`

“None”

If a function returns “None” you will generally have trouble using that value in your code, unless you use it for error checking. Note: not checking whether your function returned “None” is a very common error, and can make debugging difficult. Check for that in your calling code.

In the code in your function, having

```
    return None
```

```
    return
```

And no return statement at all have the same effect - your function returns a value of None.

More on calling by value

First the function definition

```
def reverse_word (word):
```

now the code. DON'T FORGET TO INDENT!!

```
    i = 0
```

```
    reversed_word = ''
```

```
    while i < len(word):
```

```
        reversed_word = reversed_word + word
```

```
    [-(i+1)]
```

```
#     return(reversed_word)
```

```
...     word = reversed_word
```

Now the call is

```
animals = ["cat", "Australian cattle dog",  
"duckbilled platypus", "ocelot", "zebra"]
```

```
for critter in animals:
```

```
    reverse_word(critter)
```

```
    print(critter)
```

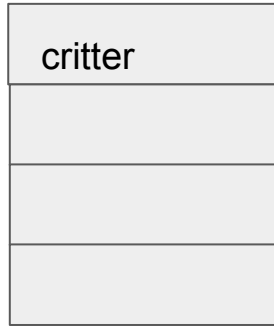
now do whatever you want to with the

reversed word in the main program

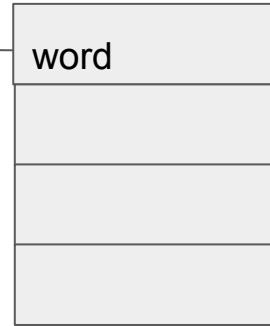
This doesn't work in Python!! (It does work in some other languages, so if you've got experience with doing this, put it out of your mind for now)

Why doesn't this work?

These are NOT the same locations in memory. The value is copied over when the function call is made. Nothing is copied back to the main program, except what's in "return"



Memory for
main program



Memory for
reverse_word

Scope of a variable

Scope of variables

Scope means where a variable can be directly seen and used

Variables defined in the main program can be seen everywhere in the program. BUT - you should only directly use them in the main program. If you need their values in a function, pass them as arguments. If you use a main program variable directly in a function, without passing it as an argument, that's called a "global variable" and it will cost you major points!!!

Variables defined in functions can only be seen and used in the functions where they are defined. These are called "local variables."

Importing modules and functions in Python

Python comes with some “builtin” functions such as `len()`, `print()`, `input()`,...

There are tons of other functions that have already been written by others, and which are free to you to use in your programming career.

- There's no need to rewrite a function if you know somebody else has already written it

You get access to that code by using the `import()` function

import()

import() tells the Python interpreter that you want access to a module that you know about, and the functions in that module

A “module” is Pythonic for a group of functions made available. Other languages might call this a “library” or a “package.”

Import random

Imports a module that contains a bunch of functions all related to the generation and management of “random” numbers

Note: the module must be present on your computer for “import” to work. If you get an error message saying the module does not exist, you’ll have to install it.

Using a function in a module

Once you have imported a module, you can use its functions in your program

```
random.randint(1,25)
```

Generates a random integer between 1 and 25, inclusive

You can use this just like any other function:

```
for i in range(10):  
    r_num = random.randint(1,25)  
    print(r_num)
```

How do you know what functions are in a module?

...and what parameters to use to call them?

This is where the ability to search the web is your friend. :-)

All the common modules are documented out there in Python-land, along with their Application Programming Interfaces (APIs)

- Which is a fancy way of saying “descriptions of how to call a function, what the parameters are, what the parameters mean and what the return values are.”